

HDOC Day-7

NAME: Yug Nirwal

SAP ID: 590033354

SUBJECT: C programming

DATE: 31-08-26

Question 1: Centripetal Force

Step 1: Problem Statement

Input: mass (m), velocity (v), radius (r)

Output: Required value

Formula: $F = (m \cdot v^2) / r$

Step 2: Test Cases

Test	Input(s)	Expected Output	Actual Output	Result
1	m=2,v=10,r=5	20.00 N	20.00 N	Pass
2	m=5,v=8,r=4	40.00 N	40.00 N	Pass
3	m=1.5,v=20,r=10	30.00 N	30.00 N	Pass

Step 3: Algorithm

1. Start
2. Read input values.
3. Apply the formula.
4. Display the result.
5. Stop

Algorithm Evaluation

The algorithm was checked using all the above test cases and produced the expected outputs. Hence, all test cases passed.

Step 4: C Program

```
#include <stdio.h>
int main(){
    float m,v,r,F;
    printf("Enter mass: ");
    scanf("%f",&m);
    printf("Enter velocity: ");
    scanf("%f",&v);
    printf("Enter radius: ");
    scanf("%f",&r);
    F=(m*v*v)/r;
```

```
printf("Centripetal Force = %f N",F);
return 0;
}
```

Step 5: Program Evaluation

The program was compiled successfully and the outputs matched the expected results for all test cases.

Step 6: Compilation & Execution Screenshots

```
(kali㉿kali)-[~/Desktop/cprog]
$ mkdir cforce

(kali㉿kali)-[~/Desktop/cprog]
$ vi cforce.c

(kali㉿kali)-[~/Desktop/cprog]
$ gcc cforce.c -lm

(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter mass= 2
Enter velocity= 10
Enter radius= 5
Centripetal Force= 20.000000N
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter mass= 5
Enter velocity= 8
Enter radius= 4
Centripetal Force= 40.000000N
```

```
(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter mass= 1.5
Enter velocity= 20
Enter radius= 10
Centripetal Force= 30.000000N
```

Question 2: Surface Tension

Step 1: Problem Statement

Input: force (F), length (L)

Output: Required value

Formula: $T = F/L$

Step 2: Test Cases

Test	Input(s)	Expected Output	Actual Output	Result
1	F=10,L=2	5.00 N/m	5.00 N/m	Pass
2	F=15,L=3	5.00 N/m	5.00 N/m	Pass
3	F=20,L=4	5.00 N/m	5.00 N/m	Pass

Step 3: Algorithm

6. Start
7. Read input values.
8. Apply the formula.
9. Display the result.
10. Stop

Algorithm Evaluation

The algorithm was checked using all the above test cases and produced the expected outputs. Hence, all test cases passed.

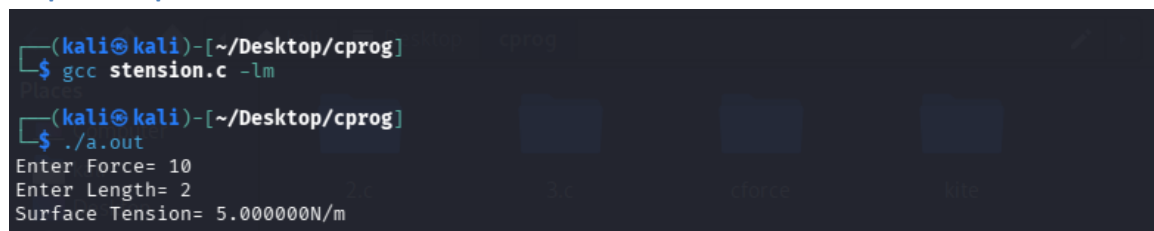
Step 4: C Program

```
#include <stdio.h>
int main(){
    float F,L,T;
    printf("Enter force and length: ");
    scanf("%f%f",&F,&L);
    printf("Enter length: ");
    scanf("%f",&L);
    T=F/L;
    printf("Surface Tension = %f N/m",T);
    return 0;
}
```

Step 5: Program Evaluation

The program was compiled successfully and the outputs matched the expected results for all test cases.

Step 6: Compilation & Execution Screenshots



The screenshot shows a terminal window on a Kali Linux system. The first command executed is `gcc stension.c -lm`, which successfully compiles the program. The second command is `./a.out`, which runs the program. The program prompts the user to enter force and length, and then displays the calculated surface tension.

```
(kali㉿kali)-[~/Desktop/cprog]
$ gcc stension.c -lm

(kali㉿kali)-[~/Desktop/cprog]
$ ./a.out
Enter Force= 10
Enter Length= 2
Surface Tension= 5.000000N/m
```

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
Enter Force= 15
Enter Length= 3
Surface Tension= 5.000000N/m
```

```
(kali@kali)-[~/Desktop/cprog]
$ ./a.out
Enter Force= 20
Enter Length= 4
Surface Tension= 5.000000N/m
```

Question 3: Refractive Index

Step 1: Problem Statement

Input: speed in vacuum (c), speed in medium (v)

Output: Required value

Formula: $n = c/v$

Step 2: Test Cases

Test	Input(s)	Expected Output	Actual Output	Result
1	c=3.00e+08,v=2.00e+08	1.50	1.50	Pass
2	c=3.00e+08,v=1.50e+08	2.00	2.00	Pass
3	c=3.00e+08,v=2.25e+08	1.33	1.33	Pass

Step 3: Algorithm

11. Start
12. Read input values.
13. Apply the formula.
14. Display the result.
15. Stop

Algorithm Evaluation

The algorithm was checked using all the above test cases and produced the expected outputs. Hence, all test cases passed.

Step 4: C Program

```
#include <stdio.h>

int main(){
    float c,v,n;
    printf("Enter speed in vacuum and medium: ");
    scanf("%f",&c);
    printf("Enter speed in medium: ");
    scanf("%f",&v);
    n=c/v;
    printf("Refractive Index = %f",n);
```

```
return 0;  
}
```

Step 5: Program Evaluation

The program was compiled successfully and the outputs matched the expected results for all test cases.

Step 6: Compilation & Execution Screenshots

```
(kali㉿kali)-[~/Desktop/cprog]  
$ mkdir rindex  
(kali㉿kali)-[~/Desktop/cprog]  
$ vi rindex.c  
(kali㉿kali)-[~/Desktop/cprog]  
$ gcc rindex.c -lm  
(kali㉿kali)-[~/Desktop/cprog]  
$ ./a.out  
Enter speed in vacuum= 3.00e+08  
Enter speed in medium= 2.00e+08  
Refractive Index= 1.500000
```

```
(kali㉿kali)-[~/Desktop/cprog]  
$ ./a.out  
Enter speed in vacuum= 3.00e+08  
Enter speed in medium= 1.50e+08  
Refractive Index= 2.000000
```

```
(kali㉿kali)-[~/Desktop/cprog]  
$ ./a.out  
Enter speed in vacuum= 3.00e+08  
Enter speed in medium= 2.25e+08  
Refractive Index= 1.333333
```

